

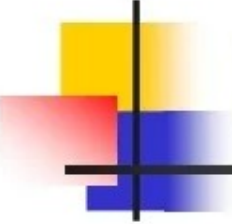
Peer to Peer Computing and JXTA



Amit Minhas

Kris Read

Ryan Peters



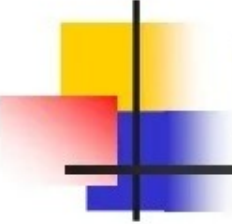
Presentation Overview

- What is Peer to Peer (P2P)?
- What is JXTA?
- JXTA Example
- JXTA Wrap-Up
- Presentation Summary



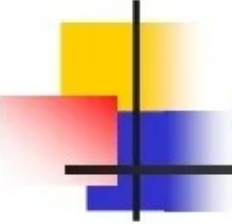
A Basic Definition of P2P

- The exchange of data (documents, MP3s, voice) between two “peers” that can act as both the client and the server.
- A basic example:
The typical telephone call.
Both “peers” act as producers (by talking)
and as consumers (by listening).



P2P and the Internet

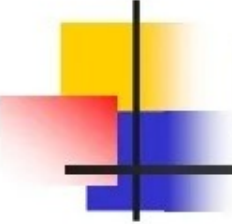
- Initially, the internet was created for the sharing of information and resources between peers.
- The paradigm shifted to a client / server architecture where there was a state of over-consumption.
- One of the goals of P2P is to return to the original intent of the internet, creating an equilibrium between production and consumption.



A Concise P2P Definition

Characteristics of P2P include the following:

- Action
- Mutual Exchange
- Information
- Services
- Directivity
- Producer / Consumer relationship
- Result



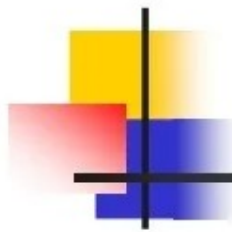
Peer to Peer Comparison

Centralized System

- More users means more server power and bandwidth is required.
- Exist only where someone is willing to invest resources.
- One location stores all the data.

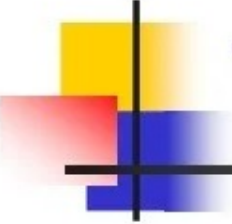
P2P System

- More users means more resources are able to share the load.
- Can exist without any person or company supporting them.
- Data must be retrieved from multiple locations.



P2P vs. Distributed Computing

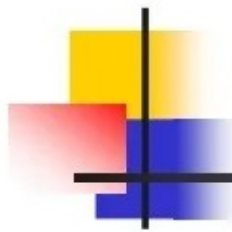
- Distributed Computing means resources are shared between nodes.
- True P2P means that there is communication between these nodes.
- P2P implies a “peer” relationship while Distributed Computing relationships may be unbalanced.
- Distributed Computing can also be P2P.



Types of P2P Communication

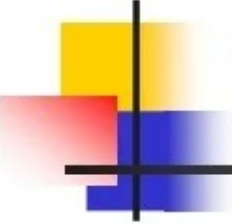
- Centralized (Wannabe P2P)
- Brokered
- Decentralized (Pure P2P)

The individual participants always contribute something of value to the network.



Familiar Examples

- Gnutella (Decentralized Peer to Peer)
 - All peers are interchangeable (act as clients and servers).
 - Search propagates through network.
- Napster (Brokered Peer to Peer)
 - Query is made to a central server, which returns a list of peers who possess the desired mp3 file.
 - The host then selects a peer and *directly* transfers the desired mp3 file.



2 Categories of P2P (1)

1) P2P Resource Products and Services

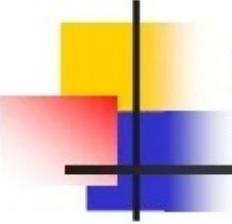
- Represents the resources that each peer owns and controls.
- Resources include raw resources such as network bandwidth, disk space and processing power.
- Centers around the information and services section of the P2P definition.



2 Categories of P2P (2)

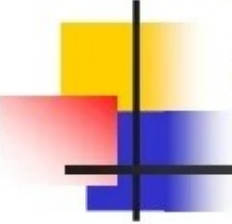
2) P2P Collaboration Products and Services

- A goal of P2P is to allow for communication between peers to accomplish business or personal goals.
- Centers more on the mutual exchange section of the definition of P2P.



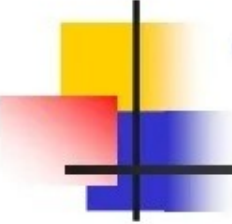
Resource Products and Services (1)

- Example: File Sharing (Freenet)
 - **What is it:** Simply an area to store documents and allow others to retrieve them.
 - **Benefit:** Allows two or more people to share products / services without the burden of having filters and media control over information.
 - **Drawback:** Files that are not popular and are not replicated often through the community will cease to exist on the network.



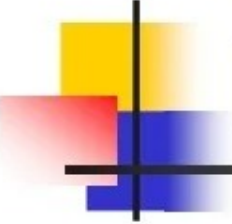
Collaboration Products and Services (1)

- Basic Collaboration Services (Groove)
 - **What is it:** A collaborative workspace that allows an individual to create P2P space and allow others to participate.
 - **Benefit:** Allows users to create instant and direct online connections with others to share and collaborate.
 - **Drawback:** It is a very ambitious project that is not complete. There is a range of possibility that has not been explored yet by developers.



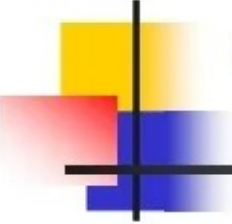
The Importance of P2P

- P2P offers the information and services that are most Important to the user.
- P2P incurs a minimal cost.
- P2P saves time.
- P2P gives the user control.
- P2P is collaborative.
- P2P can be anonymous.



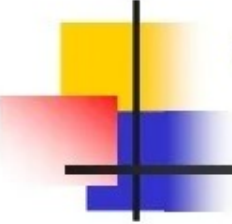
P2P Will Not Solve World Hunger

- Unpredictability
- Integrity
- Availability
- Peer and Content Control
- Discovery Problems
- Security



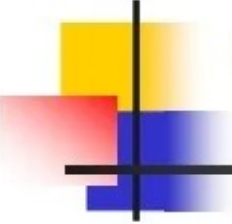
What is JXTA?

- The JXTA platform standardizes the manner in which peers:
 - Discover each other
 - Advertise network resources
 - Communicate with each other
 - Cooperate with each other to form secure peer groups



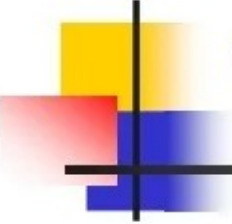
What is JXTA?

- The JXTA platform standardizes the manner in which peers:
 - Discover each other
 - Advertise network resources
 - Communicate with each other
 - Cooperate with each other to form secure peer groups



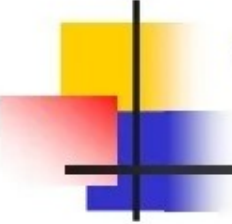
What is JXTA?

- The JXTA platform standardizes the manner in which peers:
 - Discover each other
 - Advertise network resources
 - Communicate with each other
 - Cooperate with each other to form secure peer groups



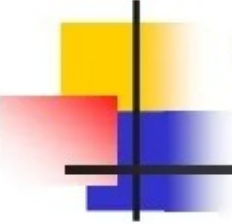
What does JXTA mean?

- Pronounced “Juxtapose” or “Juxta” for short (not an acronym).
- Peer to peer computing is “juxtaposed” to the traditional and hierarchical client-server model.
- P2P is about putting things next to each other.



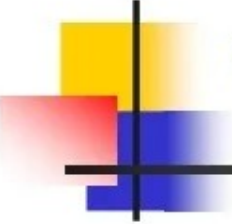
Who is responsible for JXTA?

- JXTA started as a Sun Microsystems Research Project in April 2001.
- JXTA was spearheaded by Bill Joy, chief scientist at Sun.
- JXTA.org (Project JXTA) extends the research project under the Apache software license (open source).



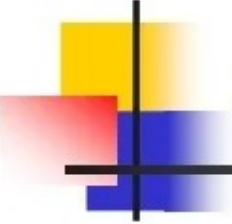
Why JXTA?

- P2P Applications are breaking new ground Sun expects more in the future.
- Current P2P programs usually address only a single function.
- Current P2P programs typically run on only a single platform (negating the usefulness of P2P).
- Most P2P applications are unable to directly share data with similar (not identical) applications (Kazaa/Gnutella, ICQ/AIM/MSIM).



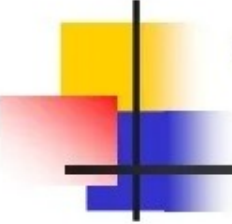
What is JXTA for?

- JXTA creates a common platform for P2P Applications.
- JXTA makes development simple, fast and easy (in theory).
- JXTA allows developers to focus on the content of their applications.



What Programming Language and Protocol?

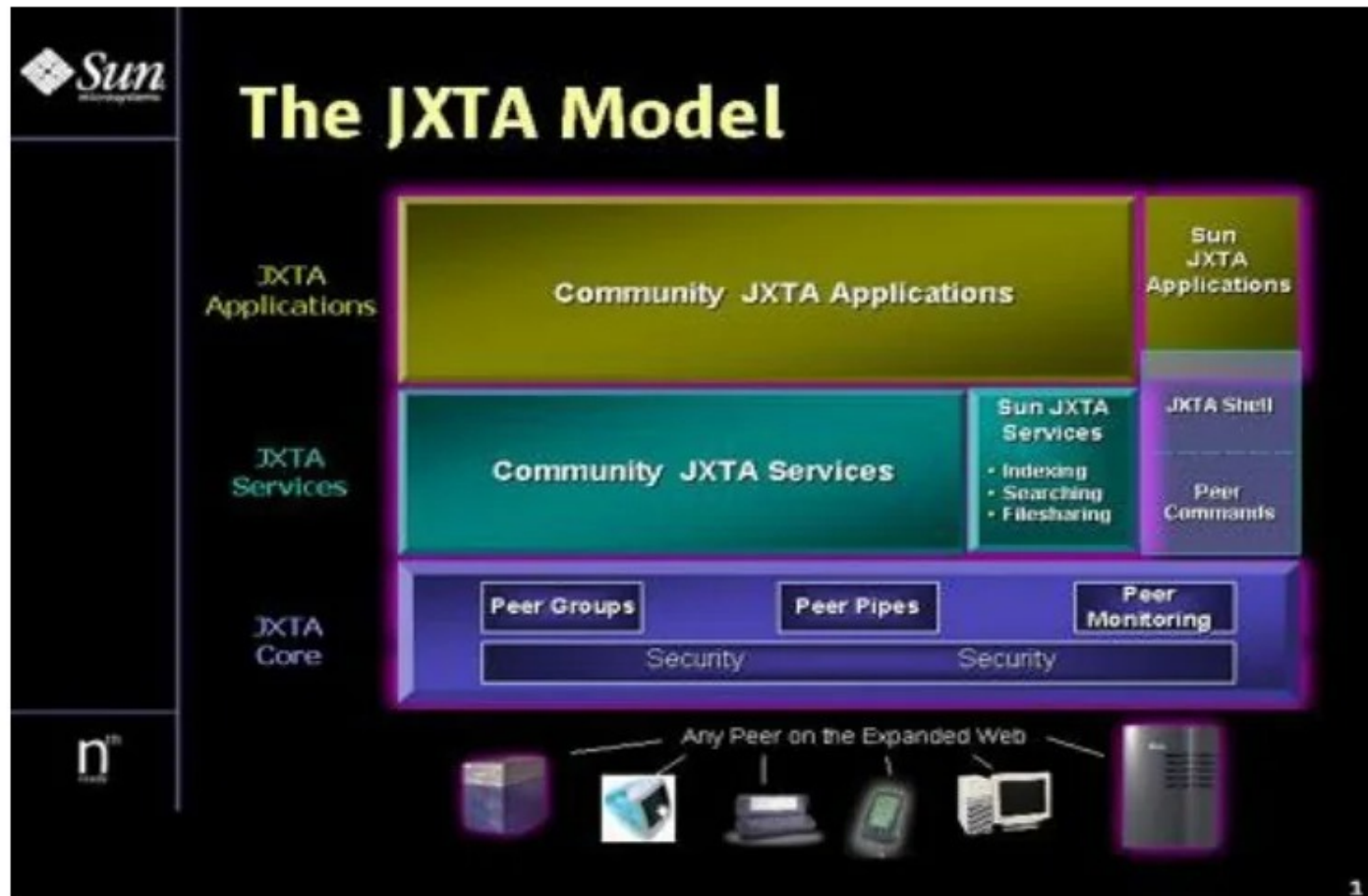
- JXTA is independent of programming languages. It is a framework, not an API.
- JXTA APIs are available for C/C++, Java, Perl, Python, Ruby, Objective C, Smalltalk and more.
- JXTA is independent of transport protocols.
- JXTA can be implemented on top of TCP/IP, over HTTP, using Bluetooth, or anything else!

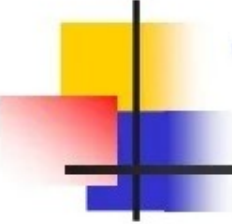


What is the state of JXTA?

- The API code that is available is early release or prototype quality.
- Code is changed (sometimes improved) by jxta.org community members.
- It is considered to be usable code, with a caution that it has not been through quality assurance processes (other than peer review).
- None of JXTA is supported by Sun.

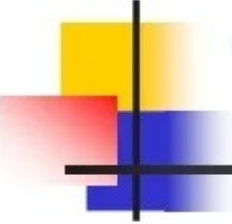
The JXTA Model (IMAGE)





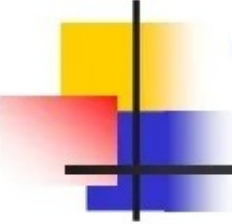
The JXTA Model

- Applications:
 - What developers build using the JXTA framework.
 - Includes P2P instant messaging, entertainment content management and delivery, email systems, distributed auction systems, and more.
 - The boundary between services and applications is not rigid.



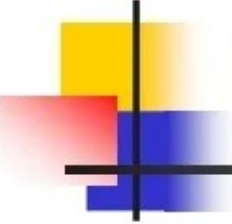
The JXTA Model

- Applications:
 - What developers build using the JXTA framework.
 - Includes P2P instant messaging, entertainment content management and delivery, email systems, distributed auction systems, and more.
 - The boundary between services and applications is not rigid.



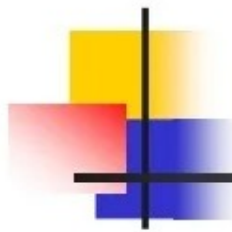
The JXTA Model

- Applications:
 - What developers build using the JXTA framework.
 - Includes P2P instant messaging, entertainment content management and delivery, email systems, distributed auction systems, and more.
 - The boundary between services and applications is not rigid.



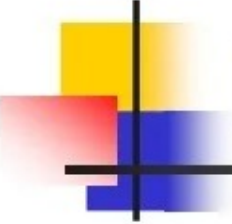
Peers

- A peer is a single participant in the JXTA network.
- Peer is referred to by an XML advertisement.
- Peer is identified by a unique peer ID.
- Each peer belongs to one or more peer groups.
- A peer can be a server, PC, PDA, some manufacturing or medical equipment, even a cellular phone.



Peer Groups

- Peers cooperate and function similarly and under a unified set of capabilities and restrictions.
- Membership policies and methods of authentication are not defined in any single fashion, but delegated to the group itself.
- When a peer joins a JXTA group, it can query that group for anything it may want, such as a service or another peer.



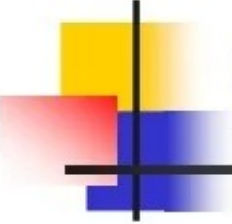
Peer Pipes

- Communication channels similar to the Unix command-line concept of piping.
- Pipes provide the primary channels for communication.
- Unidirectional (can be coupled and uncoupled between peers dynamically)
- Pipes in Project JXTA are indiscriminate they will support binary code, data strings, Java technology-based objects, and/or applets.



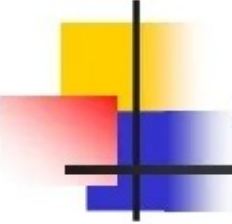
Peer Monitoring

- Handles authentication/authorization of peers.
- Important for peer **metering**.
 - Peers may want to limit the availability of their services
 - Peers may want to track the usage of their services



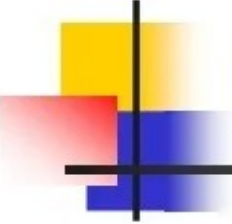
Services

- A service is a capability running inside a peer group.
- The service may be running on all peers, or just a few within the group.



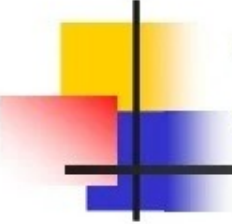
Advertisements

- An advertisement is a description of a particular pipe or service.
- Rendezvous points on the network are used by JXTA to cache advertisements.
- JXTA can use IP multicasting to find other peers on the same physical subnet.



Messages

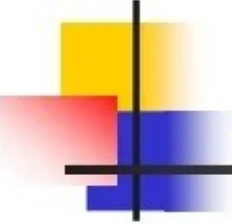
- All communication in the JXTA network is achieved by sending and receiving messages.
- JXTA messages adhere to a standard format, which is key to interoperability.
- Standard messages are defined using XML



XML format

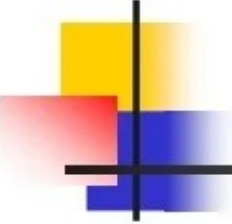
An example of an Advertisement in XML.

```
<?xml version="1.0"?>
  <!DOCTYPE jxta:PipeAdvertisement>
  <jxta:PipeAdvertisement>
    <id>
      jxta://59616261646162614A787461503250333DA9279761254559B
      209922
      BACA69E190000000000000000000000000000000000000000000000000000000
      00000000 00000000301 </id>
    <name> JxtaTalkUserName:rael </name>
  </jxta:PipeAdvertisement>
```



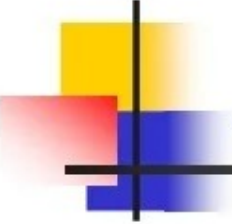
JXTA Protocols

- The JXTA platform is defined by six networking protocols.
- All six protocols needs not be implemented by every peer.
 - Peer Discovery Protocol (PDP).
 - Peer Resolver Protocol (PRP).
 - Peer Information Protocol (PIP).
 - Peer Membership Protocol (PMP).
 - Pipe Binding Protocol (PBP).
 - Peer Endpoint Protocol (PEP).



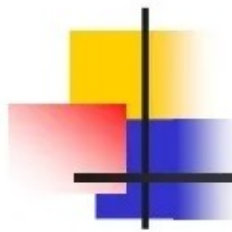
JXTA Applications

- Toys and Tutorials are available.
- Development of services comes first, applications will become common later.
- At a conference in December, Sun presenters had no knowledge of any real-world JXTA applications in use yet.



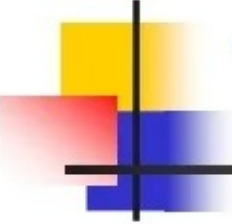
JXTA Security

- Credentials
 - Just like in PGP, the JXTA network is used to exchange public keys among its members.
 - Peer group policy may permit some members to be more trusted and thus more privileged than others.
 - Plug in modules allow JXTA policies that require authorization using well-known certificate authorities (exchange of X509v3 CA signed certificates).
- For high-level security requirements like end-to-end encryption, a security framework needs to be built on top of JXTA.



JXTA Firewall Routing Services

- When a peer joins the network from behind a firewall, it can find another peer who will perform a routing service to bypass the firewall.
- Applications will not be aware of this firewall traversal.
- Firewalls were a major limitation of early P2P systems.



The JXTA Shell

- The JXTA Shell is considered a reference peer service.
- The JXTA Shell enables users to interact with the JXTA platform through a command-line interpreter.
- Debugging and Development would be much more difficult without this tool.
- The JXTA Shell is not part of the JXTA core.

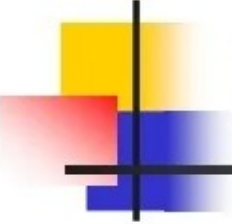


Demo of the JXTA Shell

Please be patient while we show you the basic operation of the JXTA Shell (time permitting).

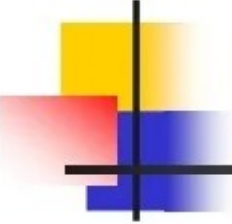
JXTA Java API Walkthrough and Application Demo





Demo Trouble?

- Although the JXTA framework does not depend on the internet, the current Java API implementation does include hard-coded internet functionality (such as centralized Rendezvous Peer lookups).
- We can show you a simple demo using a localized IP address (we think).
- We have simplified the code shown here so that we can show you the obvious syntax.



Many Many Classes

- Our **very simple** demo needs the following JXTA classes:

net.jxta.platform.Application

net.jxta.document.Advertisement

net.jxta.document.AdvertisementFactory

net.jxta.document.StructuredDocumentFactory

net.jxta.document.StructuredTextDocument

net.jxta.document.StructuredDocument

net.jxta.document.MimeMediaType

net.jxta.document.Element

net.jxta.document.StructuredDocumentUtils

net.jxta.endpoint.Message

net.jxta.pipe.PipeService

net.jxta.id.IDFactory

net.jxta.id.ID

net.jxta.pipe.InputPipe

net.jxta.peergroup.PeerGroup

net.jxta.peergroup.PeerGroupID

net.jxta.peergroup.PeerGroupFactory

net.jxta.protocol.PeerGroupAdvertisement

net.jxta.protocol.ModuleSpecAdvertisement

net.jxta.protocol.ModuleClassAdvertisement

net.jxta.protocol.PipeAdvertisement

net.jxta.discovery.DiscoveryService

net.jxta.exception.PeerGroupException

net.jxta.impl.peergroup.Platform

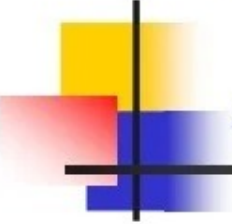
net.jxta.impl.peergroup.GenericPeerGroup

net.jxta.platform.ModuleClassID



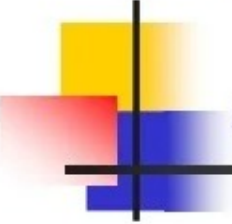
Producer: Creating the Objects

1. Create a Peer Group.
`group = PeerGroupFactory.newNetPeerGroup();`
2. Get the Advertisement for the Group.
`groupAdvertisement =
 group.getPeerGroupAdvertisement();`
3. Create a Discovery Service.
`disco = group.getDiscoveryService();`
4. Create a Pipe Service.
`pipes = group.getPipeService();`



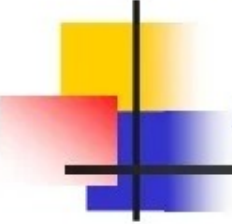
Building our Peer Existence Advertisement

1. Create an Advertisement for our Service.
`mcadv = AdvertisementFactory.newAdvertisement(type);`
2. Set the name of the advertisement.
`mcadv.setName("JXTAMOD:JXTA-EX1");`
3. Set a description for the advertisement.
`mcadv.setDescription("Example advertisement");`
4. Give the advertisement a unique ID.
`mcID = IDFactory.newModuleClassID();`
`mcadv.setModuleClassID(mcID);`
5. Publish the new Advertisement in our Discovery Service.
`disco.publish(mcadv, DiscoveryService.ADV);`
`disco.remotePublish(mcadv, DiscoveryService.ADV);`



Building our Service Advertisement

1. Create an Advertisement for the Service (just like before).
`mdadv = AdvertisementFactory.newAdvertisement(type);`
2. Set the name of the Advertisement (just like before).
`mdadv.setName("JXTASPEC:JXTA-EX1");`
3. Set a version for the Advertisement (many properties).
`mdadv.setVersion("Version 1.0");`
4. Give the Advertisement a unique ID.
`mdadv.setModuleSpecID(IDFactory.newModuleSpecID());`

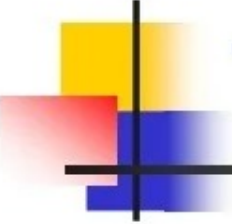


Building our Service Advertisement (2)

1. Create a Pipe Advertisement for the Service Advertisement. The steps are the same as the previous two advertisement types shown.
2. Store the Pipe Advertisement in the Service Advertisement. First we must store the Pipe Advertisement as an XML document, then add that document to the Service Advertisement!

```
paramDoc = pipeadv.getDocument(  
    new MimeMediaType("text/xml"));  
mdadv.setParam((StructuredDocument) paramDoc);
```
3. Publish the Service Advertisement in the Discovery Service.

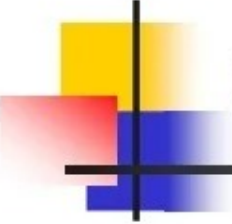
```
disco.publish(mdadv, DiscoveryService.ADV);  
disco.remotePublish(mdadv, DiscoveryService.ADV);
```



The Service in Action

1. Now that our DiscoveryService is advertising a Pipe, we should listen for messages on that Pipe.
`Message msg = myPipe.waitForMessage();`
2. The producer and consumer must agree on Tag Names for their messages.
`String myMessageString = msg.getString("DataTag");`

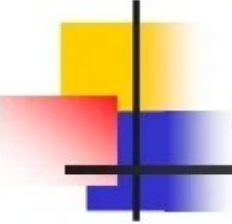
That's all! Now for the consumer code...



Consumer: Creating the Objects

Note: This is the same as our Producer code!

- Create a Peer Group.
`group = PeerGroupFactory.newNetPeerGroup();`
- Get the Advertisement for the Group.
`groupAdvertisement =
 group.getPeerGroupAdvertisement();`
- Create a Discovery Service.
`disco = group.getDiscoveryService();`
- Create a Pipe Service.
`pipes = group.getPipeService();`



Looking up a Service Advertisement

1. First check the Local Cache, a store of previously found Advertisements kept by the JXTA Framework.
`Enumeration enum = disco.getLocalAdvertisements(
 DiscoveryService.ADV, "Name", "JXTASPEC:JXTA-EX1");`
2. Send a remote discovery request searching for the service advertisement. Results end up in the Local Cache!
`disco.getRemoteAdvertisements(
 DiscoveryService.ADV, "Name", "JXTASPEC:JXTA-EX1",1);`
3. If we're lucky we find the Service Advertisement.
`ModuleSpecAdvertisement mdsadv = enum.nextElement();`

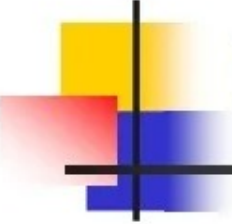


Finding the Pipe Advertisement

1. In a reversal of our producer's behavior, we must extract the Pipe Advertisement from the Service Advertisement.

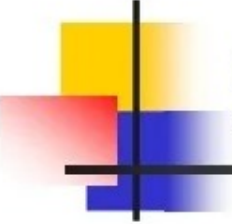
```
paramDoc = mdsadv.getParam();  
Enumeration elements =  
    paramDoc.getChildren("jxta:PipeAdvertisement");
```
2. Because there can be multiple parameters (Advertisements) within a Service Advertisement, we must choose one.

```
Enumeration e = (elements.nextElement()).getChildren();  
TextElement elem = elements.nextElement();  
String nm = elem.getName();  
pID = elem.getTextValue();  
pType = elem.getTextValue();
```



Using the Pipe Advertisement

1. Create a new Pipe Advertisement.
`pipeadv = AdvertisementFactory.newAdvertisement(type);`
2. We want our Advertisement to use the information extracted from the Service Advertisement we found.
`pipeadv.setPipeID( IDFactory.fromURL(new URL(pID) ));`
`pipeadv.setType(pType);`
3. We now have a PipeAdvertisement that we can use!
4. The next step is to construct an Output Pipe from this Advertisement.
`myPipe = pipes.createOutputPipe(pipeadv, timeout);`



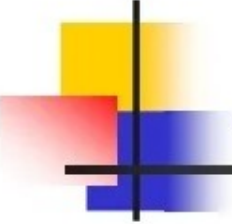
Sending a Message

1. We can create a simple Message object to send over the Pipe. This object is really represented in XML when sent.
`Message msg = pipes.createMessage();`
2. Remember that our Tag Name must be the same between producer and consumer if we are to use it.
`msg.setString("DataTag", data);`
3. Send the message using our Pipe!
`myPipe.send (msg);`

That's all there is in our Demo, but you can do much more using JXTA!

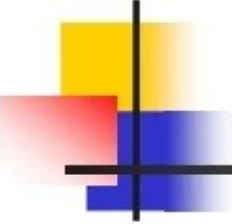


JXTA Wrap-Up



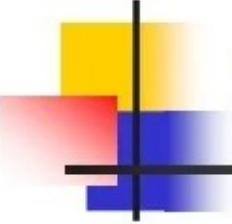
JXTA and Other Technologies

- JXTA and SOAP
 - JXTA provides additional protocols and services to implement P2P.
 - SOAP messages can be sent through Pipes.
- JXTA and JINI
 - JINI is for connecting distributed services within a **Java** network using a **centralized broker**.



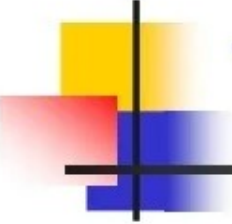
JXTA / HAILSTORM (.NET)

- Both an open, platform and language independent framework for peer interaction and interoperable services.
- Microsoft focuses on integration with its Windows operating system and associated devices.
- Hailstorm uses the Microsoft Passport service. (A centralized authentication and presence management system).
- Hailstorm content is managed and defined conceptually at a centralized source (Microsoft).



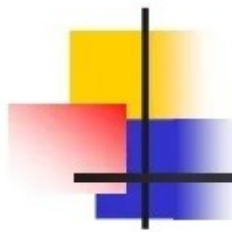
Criticism!

- JXTA is new, rapidly changing and undocumented.
- Peer 2 Peer is being fueled by Napsterism.
- Well-defined protocols can squash creativity in a new research area.
- Reliance on XML forces the hand of developers.
- JXTA is too late, there are already popular P2P Applications out there.
- Since JXTA is in its infancy, it is still too early to judge the success of JXTA.



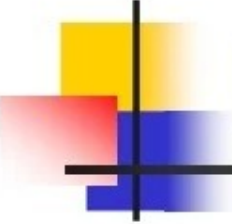
The Future of JXTA

- JXTA in J2SE Micro Edition
- Further expansion of the Java implementation.
- Implementation in other languages (C/C++/C#, Perl, Python, etc).
- Industry acceptance



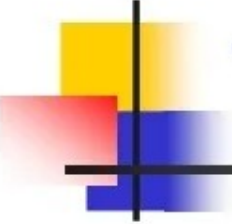
Presentation Summary

- P2P: Simply the exchange of data between two peers who act as client and as server, but the definition is open to different interpretations (De-Centralized, Brokered)
- JXTA: A Standard Platform on which we can develop P2P applications.



References

- Moore, Dana and John Hebler. "Peer to Peer." Ed. Andrew Saff. McGraw-Hill. Berkeley, California, USA. 2002.
- <http://spec.jxta.org/project/www/docs/DomainFAQ.html>
- <http://spec.jxta.org/v1.0/docbook/JXTAProtocols.html>
- <http://download.jxta.org/servlets/DomainHome>
- <http://www.javaworld.com/javaworld/jw-10-2001/jw-1019-jxta.html>
- <http://www.onjava.com/pub/a/onjava/2001/04/25/jxta.html>
- <http://www.onjava.com/pub/a/webservices/2002/03/12/jxta.html>
- <http://www.openp2p.com/pub/a/p2p/2001/06/11/ligong.html>
- <http://www.openp2p.com/pub/a/p2p/2001/06/05/jxtachat.html?page=1>
- http://www.openp2p.com/pub/a/p2p/2001/04/25/jxta_position.html
- <http://www.openp2p.com/pub/a/p2p/2001/06/06/jxtasearch.html>
- <http://instantp2p.jxta.org/servlets/ProjectHome>
- <http://java.sun.com/features/2001/02/peer.html?frontpage-banner>
- www.sce.carleton.ca/netmanage/papers/WhiteJuly2001.ppt



Thank You For Listening

Amit Minhas

Kris Read

Ryan Peters